

EV Charging Platform

Solution Design Decisions

Principal AI Engineer — Solution Design Challenge

1. Purpose & Approach

This document captures the architecture, entity model, and engineering trade-offs behind the backend service built for this exercise. Per the brief, the goal is not full completeness but demonstrated judgement: what was built, what was simplified, what was deferred, and why.

The document is structured in three layers. First, the platform vision: the high-level, target-scale architecture this domain naturally grows into. Second, the exercise design: what was actually built, scoped deliberately to the available timeline. Third, the adaptation path: an explicit mapping showing how the exercise design evolves into the vision architecture as real demand justifies it. Section 16 closes the loop with a line-by-line traceability matrix checking every requirement in the original brief against where it's covered. Section 2, immediately below, grounds all of this in why the platform exists before any architecture is discussed.

2. Business Context & Use Cases

Before any architecture: who is this platform for, and what do they actually come to it to do. This section uses a System Context diagram to fix the platform's boundary and its relationships to the people and external systems around it, then lists the primary business use cases those relationships produce.

2.1 System Context

At this level, the platform is a single box. The diagram deliberately does not show internal architecture (that's Section 3 onward) — only who and what sits outside the boundary and why they interact with it.

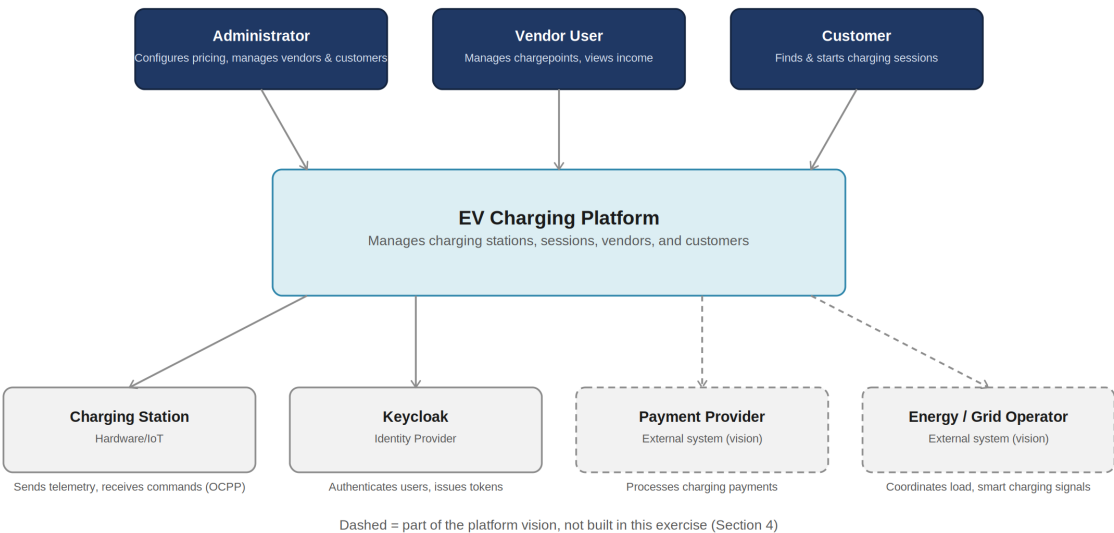


Figure 1 — System Context: three actor types drive the platform from above.

Three things worth calling out about this boundary:

- The platform sits between two very different worlds: people using REST APIs on one side, and physical hardware speaking OCPP on the other. That asymmetry is the reason the internal architecture (Section 3) splits into an API Gateway path and a Device Gateway path rather than treating all traffic uniformly.
- Keycloak is drawn as an external system rather than inside the platform boundary on purpose — it's a dependency the platform trusts for identity, not a component the platform owns (Section 3.2).
- Payment Provider and Energy/Grid Operator are real relationships in the platform's target end-state, not in this exercise. They're included here so the business context is honest about where the platform is ultimately headed, with the exercise's actual boundary made explicit in Section 4.

2.2 Primary Business Use Cases

Every relationship in Figure 1 resolves into concrete use cases. These map directly onto the entity model's capability bullets (Section 5) and the traceability matrix (Section 16) — this table is the business-level summary; those sections are where each one is actually designed and verified:

Use Case	Primary Actor	Where It's Designed
Locate an available charging station and its price	Customer	Section 5 (Chargepoint), Section 7 (API)
Start a charging session, identified by RFID, plate, or manual selection	Customer	Section 5.5, Section 3.4.2
Monitor charging progress and stop a session	Customer / Platform	Section 5 (ChargingSession), Section 7
View charging session history and monthly totals, including the current partial month	Customer	Section 7 (GET /customers/{id}/sessions?month=)
Register and de-list vehicles	Customer	Section 5.1, Section 7
Add, update, and remove chargepoints	Vendor	Section 7 (chargepoint CRUD)
View income and charging activity, including current-month and per-chargepoint breakdowns	Vendor	Section 7 (GET /vendors/{id}/chargepoints/{cpId}/income)
Generate a session report for a chargepoint on a given date	Vendor	Section 7 (GET /chargepoints/{id}/sessions/report)
Manage vendor-owned resources according to assigned permissions	Vendor User	Section 5.4, Section 6
Sign up a new vendor and invite its first vendor user	Administrator	Section 7 (POST /vendors)
Set the markup added to a vendor's unit price	Administrator	Section 3.1 (Pricing & Billing), Section 10.3
View income over a period, optionally filtered by vendor	Administrator	Section 7 (GET /admin/income)
Reset credentials for a user where appropriate	Administrator	Section 7 (POST /users/{id}/reset-credentials)
Search sessions, customers, and vehicles with partial-match terms	Administrator	Section 9 (full-text search)

These fourteen use cases are exhaustive against the brief (Section 16 confirms this bullet by bullet) and map cleanly onto three of the four actor types in Figure 1. The fourth — the charging station itself — doesn't drive a “use case” in the traditional sense; it's the source of the telemetry events described in Section 3.4.3, which is why it's modelled as an external system rather than a person in the C4 diagram.

3. Platform Vision: High-Level Architecture

Where Section 2.1 showed the platform as a single box (System Context), this section opens it up one level. Every box below is a container — an independently runnable or deployable unit with its own technology choice — and the same actors and external systems from Section 2.1 (Administrator, Vendor User, Customer, Charging Station, Keycloak, Payment Provider) reappear here, now shown crossing into or out of an explicit system boundary rather than connecting to one opaque box.

The same domain naturally extends into a multi-sided EV charging network platform in the style of ChargePoint, Tesla Supercharger, or Shell Recharge — connecting operators, vendors, customers, vehicles, payments, energy providers, and IoT devices. The diagram below is the target end-state architecture this platform grows toward, not what was built for this exercise.

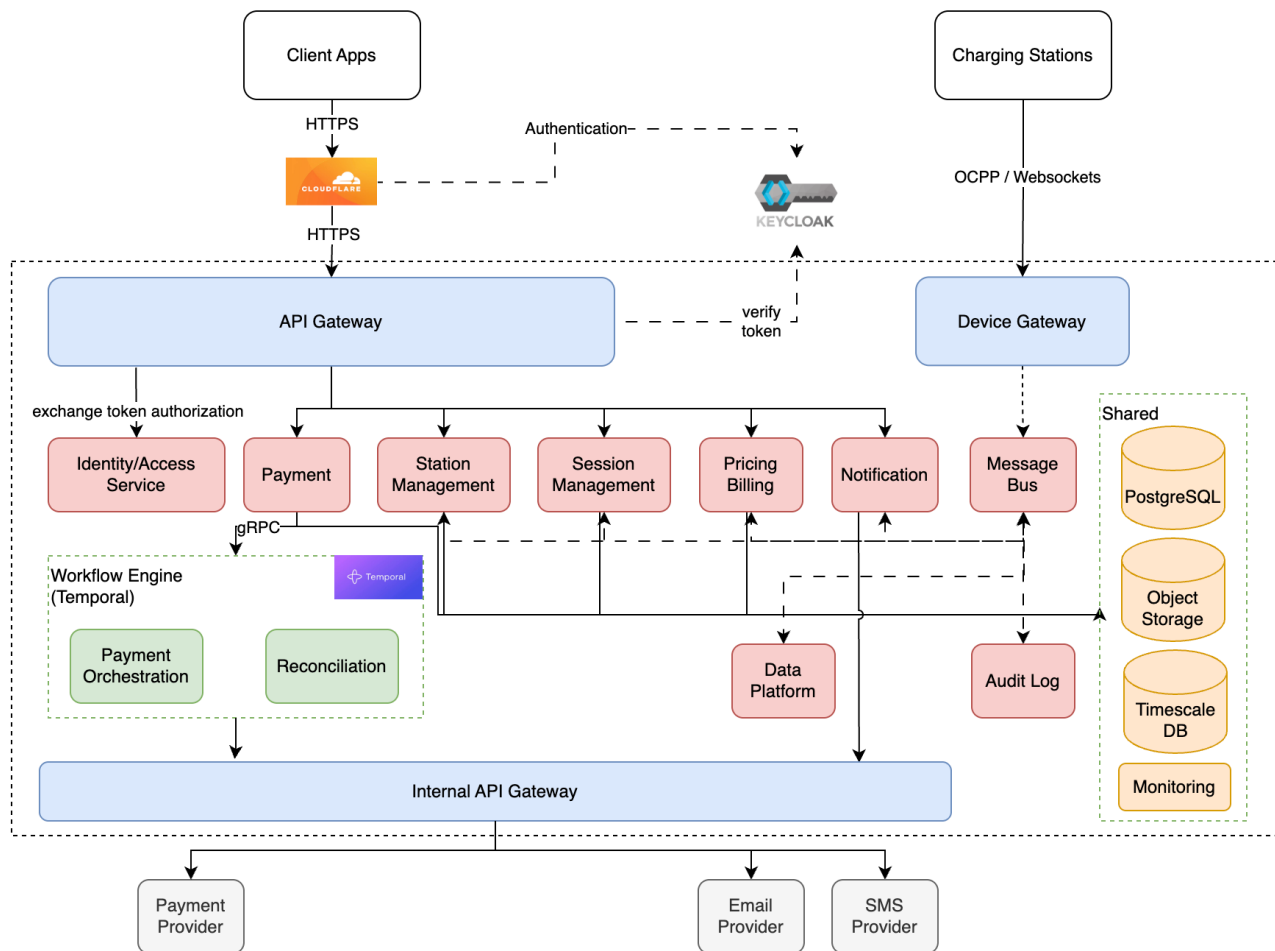


Figure 2 — EV Charging Platform as a software system.

The diagram deliberately shows two distinct integration paths rather than one shared bus, because they solve different problems:

- **API Gateway → services (direct, synchronous)** — client requests (mobile app, admin dashboard, vendor portal) are request/response by nature: “create this vendor,” “show me this session.” The gateway calls the relevant service directly over REST; there is no reason to route a synchronous read or write through an event bus, and doing so would only add latency and an unnecessary point of failure.
- **Device Gateway → Message Bus → Station Mgmt, Session Mgmt (asynchronous)** — charger telemetry (Heartbeat, StatusNotification, MeterValues, Start/StopTransaction) is fundamentally event-shaped and one-to-many in effect: a single OCPP message needs to update station health and progress an active session, without the

charger itself waiting on either. The event bus is scoped specifically to this integration, not used as a general-purpose message bus for every service.

3.1 Target Service Decomposition

Domain-driven decomposition gives each bounded context clear ownership of its own data. Client-facing services are reached directly through the API Gateway; Station Mgmt and Session Mgmt additionally consume device events from the Domain Event Bus:

Service	Core Responsibility	Key Entities
Identity & Access	Authorization: RBAC, fine-grained permissions, role-to-resource mapping. Authentication itself is delegated to an external IdP (see Section 3.2)	Roles, Permissions, UserProfile (linked to IdP subject)
Station Mgmt	Station registry, connectors, health, firmware metadata	Stations, Connectors, Vendors, Firmware
Session Mgmt	Charging session lifecycle, metering, state machine	Sessions, Transactions, MeterValues
Pricing & Billing	Tariff rules, cost calculation, invoicing	Tariffs, PriceRules, Invoices
Payment Processing	Payment orchestration, provider integration, PCI scope	Payments, Refunds, PaymentMethods
Notification	Multi-channel delivery, templates, preferences	Notifications, Templates, Channels
Device Gateway	OCPP protocol handling, message routing, device auth	DeviceConnections, MessageQueue

This decomposition is deliberately close to the module boundaries already used inside the exercise's monolith (Section 5 onward) — the intent is that today's internal module split is tomorrow's service split, not a different shape entirely.

3.2 Authentication vs. Authorization: Where the Boundary Sits

Keycloak (or an equivalent OIDC provider) owns authentication — verifying credentials, MFA, social/enterprise login, and issuing the token. Identity & Access owns authorization — interpreting that token's identity against this platform's own roles and permissions to decide what the caller may do. This split is deliberate rather than incidental: authentication is a solved, standardised problem that a generic IdP handles well precisely because it knows nothing about this domain, while authorization here is domain-specific and changes whenever the business rules do (a new vendor-permission type, a new admin capability). Keeping them separate means adding a permission never touches IdP configuration, and rotating or replacing the IdP never touches authorization logic.

Concretely: Keycloak issues a token carrying a stable subject identifier and, at most, a coarse account type (admin, vendor user, customer). Identity & Access holds the fine-grained permission model — the vendor-user permission set from Section 5.4, tenant scoping from Section 6 — keyed off that subject identifier, not duplicated into the IdP. The API Gateway's Resource Server role (Section 8) is the seam between the two: it validates the token came from a trusted issuer, then passes the validated identity through for Identity & Access and each service's own authorization checks to interpret.

3.3 Component Descriptions

Every box in Figure 2 has a distinct, single-sentence job. Grouped by layer, each with the technology it's actually built (or planned to be built) in:

Entry Points, Gateways & Integration

Component	Responsibility	Technology	Talks To
Client apps	Mobile app, vendor portal, and admin dashboard — anything a person uses to interact with the platform. Not built in this exercise (no UI required).	External (not built)	Keycloak (login), API Gateway (everything else)
Charging stations	Physical AC/DC/ultra-fast chargers in the field. Speak OCPP, not REST — they are hardware, not API clients.	External (hardware)	Device Gateway (OCPP over WebSocket)
Keycloak	Owns authentication: credential verification, MFA, social/enterprise login, and JWT issuance. Knows nothing about this platform's domain — deliberately (Section 3.2).	External (OIDC provider)	Client apps (login flow), API Gateway (token trust)
API Gateway	Single entry point for synchronous client traffic. Terminates TLS, validates JWTs as an OAuth2 Resource Server, and routes each request directly to the owning service. Carries no business logic.	Spring Cloud Gateway / Kong / WSO2	Keycloak (token validation), all six domain services directly
Device Gateway	The protocol boundary for physical hardware. Handles OCPP connection lifecycle (BootNotification, Heartbeat, Authorize) and translates raw OCPP frames into normalised domain events, so no other service has to speak OCPP.	Spring Boot + WebSocket / Native IoT	Charging stations (OCPP), Domain Event Bus (publishes events)
Message Bus	A narrow, purpose-built integration layer — not a general message bus for the whole platform. Carries charger telemetry from the Device Gateway to exactly the two services that need it. In-process today; Kafka if device volume later demands durable, replayable delivery.	In-process → Kafka later	Device Gateway (consumes from), Station Mgmt + Session Mgmt (publishes to)

Domain Services

Component	Responsibility	Technology	Talks To
Identity & Access	Authorization only (Section 3.2): RBAC, fine-grained permissions, tenant/vendor scoping, keyed off Keycloak's subject identifier.	Spring Boot	API Gateway (direct); consulted by every other service for authorization decisions
Station Mgmt	Station and chargepoint registry: ownership, connectors, unit price, availability, firmware metadata. Updates health/status from device telemetry.	Spring Boot	API Gateway (direct); Domain Event Bus (StatusNotification, Heartbeat)
Session Mgmt	Charging session lifecycle: start, stop, metering, state. Owns the transactional record of what happened during a charge.	Spring Boot	API Gateway (direct); Domain Event Bus (StartTransaction, MeterValues, StopTransaction)
Pricing & Billing	Tariff rules, markup/rate calculation, invoice generation. In the exercise, only the rate-calculation half of this is built (Section 15.1).	Spring Boot	API Gateway (direct); called by Session Mgmt for the effective rate
Payment Processing	Payment orchestration and provider integration (e.g. Stripe, Adyen), kept out of PCI scope for the rest of the platform via tokenisation.	Spring Boot	API Gateway (direct); triggered by session-completed / invoice-

			generated events; Internal API Gateway (outbound); Temporal Cluster (orchestration)
Notification	Multi-channel delivery (push, SMS, email) driven by domain events — session complete, payment failed, station offline.	Spring Boot	API Gateway (direct, for preferences); triggered by events from other services

Outbound Integration & Workflow Orchestration

Two components exist purely to serve Payment Processing's external-facing responsibilities, and neither is a domain service in its own right:

Component	Responsibility	Technology	Talks To
Internal API Gateway	Egress point for calls out to external providers — isolates services from third-party API specifics (retries, circuit breaking, credential rotation) behind a stable internal interface. The inverse of the (inbound) API Gateway: this one exists so services never hold a direct dependency on a vendor's SDK.	Spring Cloud Gateway / Kong / WSO2	Payment Processing (outbound); Payment Provider (external)
Payment Provider	Third-party payment processor (e.g. Stripe, Adyen) that actually moves money. External system (Section 2.1), not owned by this platform — the same box shown in the C4 context diagram.	External (For example: Stripe/Adyen ..)	Internal API Gateway
Temporal Cluster	Durable workflow orchestration engine for the payment and reconciliation process — gives each step of the session-completion chain (Section 3.4.4) automatic retry, durable state that survives a crash, and visibility into exactly where a stuck workflow is stuck. Only justified once a workflow spans multiple services with real failure modes to recover from (Section 14).	Temporal	Payment Processing (primary); coordinates with Pricing & Billing and Notification for the same workflow

Data Layer

Component	Responsibility	Technology	Talks To
Data Platform	Aggregates data out of all six domain services into read models for reporting, analytics, and search — so operational services stay lean and don't carry reporting-query load.	Spring Boot (batch/stream) / Cloud Native	All six domain services (consumes from)
PostgreSQL	System of record, one logical schema per module (Section 10.1). Source of truth for every domain service's own data.	PostgreSQL	Every domain service (its own schema only)
TimescaleDB	High-volume, time-series telemetry — MeterValues readings arrive far more often than session or station records change, and benefit from time-partitioned storage.	TimescaleDB	Data Platform (ingest and query)
Object storage (S3)	Large or binary artefacts that don't belong in a relational row: firmware images, generated invoice PDFs.	AWS S3 (or equiv.)	Station Mgmt (firmware), Pricing & Billing (invoices)
Audit log	Immutable, append-only record of business-significant mutations, retained for compliance (7 years). Written to, never queried by, the business logic.	Append-only table	Every domain service (write-only)

3.4 Architecture Flows

Four flows explain how the components in Figure 2 actually cooperate. The first two are the platform's everyday paths; the third and fourth show why the event bus and a decoupled billing step exist at all.

3.4.1 Authentication Flow

1. A client app authenticates directly against Keycloak (username/password, MFA, or social/enterprise login — entirely outside this platform's own code).
2. Keycloak issues a JWT carrying a stable subject identifier and a coarse account type (admin, vendor user, customer).
3. The client attaches that token as a Bearer credential to every subsequent API Gateway request.
4. The API Gateway, acting as an OAuth2 Resource Server, validates the token's signature, expiry, and issuer against Keycloak's public keys — it never contacts Keycloak synchronously per request.
5. **The authenticated identity is propagated to downstream services.** Each service (or a centralized Identity & Access service) performs authorization by mapping the authenticated user to the platform's own roles, permissions, and business rules. Authorization decisions are based on the platform's permission model rather than Keycloak's internal authorization model.

3.4.2 Synchronous Client Request Flow

Example: a vendor requesting their chargepoint income breakdown.

1. **The client application sends a GET request to the API Gateway** with its Bearer access token.
2. **The API Gateway validates the JWT** (as described in Section 2.4.1), forwards the authenticated principal to the appropriate backend service, and routes the request to **Station Management** (or the service that owns the endpoint). No event bus is involved because this is a synchronous request/response interaction.
3. **Station Management authorizes the request** by evaluating whether the authenticated user has permission to access the requested vendor's resources. This authorization may involve consulting the Identity & Access module for platform roles and vendor membership before applying its own domain-specific business rules.
4. **Station Management retrieves the requested data** from its own datastore. For reporting, analytics, or other precomputed aggregates, it may instead query the Data Platform's read models or reporting views rather than recalculating results from operational data.
5. **Station Management returns the response to the API Gateway**, which forwards it to the client.

3.4.3 Charger Telemetry Flow (Event-Driven)

Example: a charging station reporting a MeterValues reading mid-session.

1. **The charging station sends an OCPP MeterValues message** over its WebSocket connection to the Device Gateway.
2. **The Device Gateway validates and translates the OCPP message** into one or more normalized domain events (for example, MeterValuesReceived) and publishes them to the Domain Event Bus. Downstream services are decoupled from the OCPP protocol and consume only domain events.
3. **Interested services consume the events independently.** For example, Session Management updates the in-progress charging session's energy consumption, while Station Management consumes operational events such as StatusNotification and Heartbeat to maintain charger health and availability.
4. **Each service processes the event and persists its own state independently.** Event publication is asynchronous, so the Device Gateway does not wait for downstream database writes before acknowledging the OCPP message. This prevents slow or temporarily unavailable downstream services from impacting charger communication.
5. Operational telemetry is stored in the appropriate telemetry store (for example, TimescaleDB), while the Data Platform consumes events or read models from the owning services to build reporting and analytical views.

3.4.4 Session Completion & Billing Flow

This flow is deliberately decoupled rather than a single synchronous chain, echoing the reasoning already captured in Section 14's decision log (cross-module transactions, payment settlement).

1. **Session Management marks a charging session as complete** (for example, after receiving an OCPP StopTransaction or a customer ending the session through the mobile application) and publishes a SessionCompleted domain event.
2. **A Temporal workflow is started** in response to the SessionCompleted event. The workflow orchestrates the remaining business process while persisting its execution state durably.
3. **The workflow invokes Pricing & Billing** through the REST APIs to calculate the final charging cost and generate an invoice.
4. **The workflow invokes Payment Processing** through the Internal API Gateway to charge the customer's stored payment method. If the payment provider is unavailable or the request times out, Temporal automatically retries according to the configured retry policy without affecting the completed charging session.
5. **As significant milestones occur** (for example, InvoiceGenerated, PaymentSucceeded, or PaymentFailed), the owning services publish domain events. Notification consumes these events to send receipts or operational alerts, while other services remain loosely coupled.
6. **If the workflow cannot complete successfully**, Temporal retains the workflow state and resumes execution when possible or escalates according to business policy. Scheduled reconciliation workflows periodically identify incomplete or failed payment workflows and retry or escalate them as appropriate

At this scale, the business process is implemented as a **Temporal Saga workflow** rather than a collection of unrelated event handlers. Temporal durably persists workflow state, enabling automatic retries, compensation, and recovery from partial failures. For example, if payment succeeds but receipt delivery is interrupted, the workflow resumes from the last completed step rather than restarting or requiring manual intervention. The same workflow engine also executes scheduled reconciliation processes, such as retrying failed payments or escalating sessions that remain incomplete beyond a defined threshold.

4. Scope of This Exercise (Timeline-Driven)

Given the exercise's timeline and scope, the implementation deliberately adopts a **modular monolith** rather than the target production architecture described above. It focuses on delivering the explicit, testable requirements defined in the brief—namely station, vendor, customer, and session management, together with authentication and full-text search.

Each business capability is implemented as an independent module with clear boundaries, well-defined interfaces, and minimal coupling. This preserves the ability to evolve the system incrementally towards the target architecture, allowing individual modules to be extracted into independent services in the future with minimal impact on the overall design.

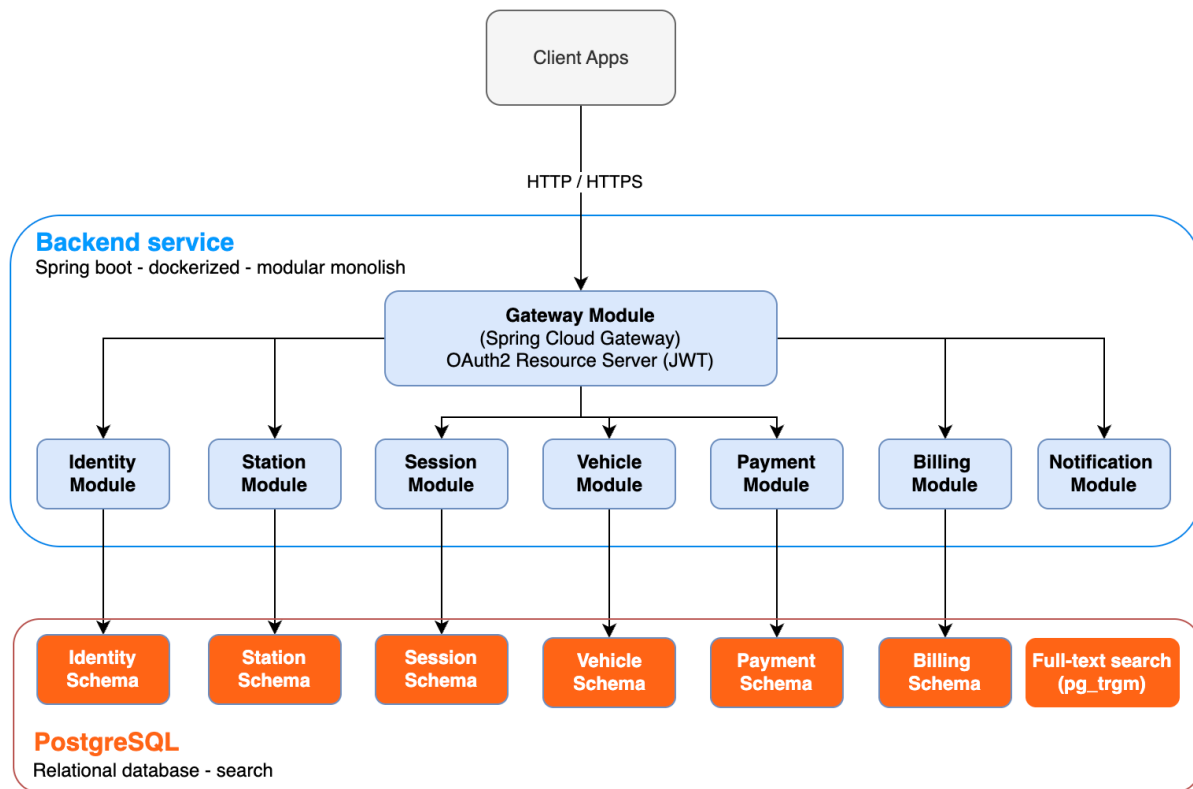


Figure 3 — What was actually built: a Spring Cloud Gateway (OAuth2 Resource Server) fronting a single dockerized Spring Boot service over one PostgreSQL instance, with the deferred vision components shown for contrast.

Four architectural decisions define this scope. Each is deliberate, not a default, and each is explained below on its own terms rather than left implicit.

4.1 Why a Modular Monolith, Not Microservices

The vision architecture (Section 3) decomposes the platform into seven services. Implementing that architecture for this exercise would introduce complexity that is not justified by the current scope and operating model: there is a single delivery team, a single deployable application, and no requirement in the brief for independent scaling, deployment, or operational ownership of capabilities such as Station Management and Session Management.

Microservices introduce distributed-system concerns that do not exist within a single process: network communication instead of direct method calls, eventual consistency instead of local transactions, service-to-service authentication, distributed observability, and independent deployment pipelines. These capabilities provide value when individual domains need to scale, evolve, or fail independently, but they also introduce operational overhead. Those trade-offs are not justified at the current stage of the platform.

The modular monolith is not a rejection of the target architecture. It applies the same domain boundaries identified in Section 3.1 while choosing a different deployment topology for the current phase. Each module maintains its own domain logic, persistence boundaries, and well-defined interfaces (Section 10), creating a structure that can evolve towards independently deployable services when future scaling, ownership, or operational requirements justify that transition (Section 15).

The decision is therefore about **deployment topology today**, not about abandoning the domain decomposition or limiting future architectural evolution.

4.2 Why Spring Cloud Gateway as an OAuth2 Resource Server

Authentication is handled at the edge through Spring Cloud Gateway configured as an OAuth2 Resource Server. The gateway is responsible for validating incoming access tokens, including signature verification, issuer validation, audience checks, and expiry validation, before forwarding authenticated requests to backend modules.

This keeps authentication concerns separate from business capabilities. Backend modules focus on authorization decisions specific to the platform, such as role-based permissions, vendor/customer ownership rules, and tenant scoping (Section 6), rather than implementing JWT parsing and token validation logic throughout controllers and services.

This is a deliberate architectural boundary that provides immediate value. Introducing or changing an external identity provider (such as Keycloak, Azure AD, or another OIDC-compatible provider) primarily affects the authentication configuration at the gateway layer, including issuer and JWKS configuration, rather than requiring authentication logic to be duplicated across application modules. Any provider-specific claim mapping can be isolated at the identity boundary.

The gateway also provides a natural location for cross-cutting concerns such as rate limiting, request tracing, audit logging, and security policies without coupling those concerns to domain logic.

4.3 Inter-Module Communication

Although the platform is implemented as a modular monolith, modules communicate through the same boundaries that would exist if they were deployed independently. A module does not access another module's internal classes, repositories, or database schema directly. Instead, interaction occurs through explicitly defined contracts.

For synchronous request/response scenarios, modules communicate through application-level interfaces exposed by the owning module. For example, Session Management may request customer or vehicle information through an Identity or Customer module interface rather than querying another module's persistence layer. This keeps ownership of business rules and data within the responsible module.

For asynchronous workflows and cross-module side effects, modules communicate through domain events. For example, when a charging session completes, Session Management publishes a `SessionCompleted` event that other modules, such as Billing or Notification, may consume independently. This avoids tightly coupled call chains and provides a natural path towards event-driven communication if modules are extracted into separate services later.

Each module owns its own persistence model. A module may expose query or command interfaces, but other modules must not bypass these boundaries by reading or writing directly to its database tables.

This approach provides the benefits of service-oriented boundaries while retaining the simplicity of in-process communication. If future requirements justify extracting modules into independent services, the existing interfaces and events become the foundation for replacing in-process calls with network APIs or message-based communication.

4.4 Cross-Module Orchestration Workflow

The scoped implementation contains one primary cross-module orchestration flow: charging session completion. When a session completes, Session Management synchronously requests the effective charging rate from the Pricing module, then persists the completed session with the calculated pricing information snapshotted as part of the session record (Section 5.2, Section 5.3).

This is intentionally implemented as a simple in-process orchestration rather than introducing a saga or workflow engine. At the current scope, the interaction involves a single downstream module within the same application boundary, with no external systems, long-running operations, or independent failure domains involved. A database transaction provides the required consistency guarantee: if pricing retrieval fails or session persistence fails, the transaction is rolled back and no incomplete session state is committed.

This approach is a deliberate trade-off. It provides strong consistency and implementation simplicity for the current modular monolith while accepting tighter coupling between the participating modules. The module boundaries and interfaces are maintained so that this interaction can later evolve into an asynchronous workflow if future requirements introduce independent services or external dependencies.

This differs from the vision-scale architecture described in Section 14, where session completion expands into a multi-step business process involving Billing, Payment Processing, and Notification. Once a workflow spans multiple services, external providers, retries, and partial failures, a synchronous transaction is no longer sufficient. At that point, a durable workflow orchestration approach such as Temporal, with persisted state and compensation handling, becomes appropriate.

The current design therefore implements the simplest reliable solution for the exercise while preserving the architectural path towards the more distributed workflow model described in the target architecture.

5. Entity Model & Relationships

The following entities represent the core business model for the MVP. Each entity boundary reflects a specific business responsibility, with explicit decisions made for areas where a simpler model could create future ambiguity.

Entity	Key Properties	Relationships	Notable Decision
AdminUser	name, email	Global scope	Not tenant-scoped. Administrators can manage platform-wide configuration and operate across all vendors.
Vendor	name, account number	1—N VendorUser; 1—N Chargepoint	Represents a charging network/operator. Pricing configuration is managed at vendor level rather than independently per chargepoint.
VendorUser	name, email, vendor	Belongs to exactly one vendor	Permissions are modelled through an assigned permission set rather than a single role flag, allowing finer-grained access control.
Chargepoint	unique identifier, group label, unit price, availability, vendor	Owned by one vendor	The stored unit price represents the vendor's base charging rate. The effective customer charging rate is calculated as base rate plus vendor-specific markup.
CustomerUser	name, email, account number, phone	1—N Vehicle; 1—N ChargingSession	Represents the charging customer. Billing and payment processing are explicitly outside the MVP scope.
Vehicle	registration plate, RFID, customer	Owned by one customer; can be de-listed / re-listed	Registration plate is not treated as a permanent identity. The same plate may be associated with a different customer after vehicle ownership changes.
ChargingSession	start, end, vehicle, chargepoint, marked-up unit rate, error code, total energy, total charged	Belongs to one vehicle and one chargepoint	The effective charging rate is snapshotted when the session starts. This preserves historical accuracy even if vendor pricing changes later.

5.1 Vehicle Identity Over Time

A vehicle registration plate is not a permanent identity because the same plate may be assigned to different owners over the vehicle's lifetime. Therefore, the Vehicle entity uses a synthetic identifier (UUID) as its primary key, with the registration plate stored as an indexed business attribute rather than a unique identifier.

Vehicle lifecycle changes are modelled through status transitions rather than hard deletion. A vehicle can be de-listed or deactivated while remaining available for historical reference, ensuring that completed charging sessions continue to reference the original vehicle record rather than becoming orphaned.

Where vehicle ownership changes need to be supported, ownership should be modelled as a time-bounded relationship rather than overwriting the current customer reference. This preserves historical accuracy by allowing past charging sessions to remain associated with the customer who owned the vehicle at the time of charging.

5.2 Monetary Precision

Charging prices require sub-cent precision because unit prices are defined in tenths of a cent. Floating-point types are therefore not suitable for monetary values due to their inability to represent decimal fractions exactly and the risk of cumulative rounding errors.

All monetary values use an explicit precision strategy throughout the system. Internally, monetary amounts are represented using integer-based units where possible (for example, storing tenths of a cent as a whole number), ensuring deterministic calculations for markup, energy consumption, and total charging cost.

Where decimal arithmetic is required, BigDecimal is used with an explicitly defined scale and rounding policy. The same precision rules apply consistently across persistence, business calculations, and API payloads to prevent rounding drift and ensure that calculated amounts remain accurate throughout the charging lifecycle.

5.3 Session Month Bucketing

Charging sessions are assigned to the calendar month in which they start. This rule ensures consistent reporting behaviour: a session that spans multiple calendar months remains attributed to the month when the charging activity began.

Rather than deriving the reporting period dynamically using date extraction functions during every aggregation query, the session stores a derived period value (for example, year_month) at creation time. This value can be indexed to support efficient monthly income, energy usage, and session volume queries without repeatedly calculating the reporting bucket at query time.

The derived field is treated as a query optimisation and reporting attribute, while the session start timestamp remains the authoritative source of time-based information.

5.4 Vendor User Permissions

The brief's phrase "according to their assigned permissions" implies a permission-based authorization model rather than a single fixed role value. The design therefore introduces an explicit permission model where capabilities such as `MANAGE_CHARGEPOINTS`, `VIEW_INCOME`, and `MANAGE_VENDOR_USERS` are assigned to a `VendorUser` through a dedicated association table.

This avoids coupling authorization behaviour to a hard-coded role enum and allows new permissions to be introduced without requiring schema changes. Roles, if introduced later, can act as permission bundles rather than replacing the underlying permission model.

Authentication identity is managed separately from application permissions. The authenticated user identity provided by the identity provider is mapped to a platform user record, and authorization decisions are made using the platform's own permission model.

5.5 Vehicle Identification at Session Start

A charging session must resolve its associated vehicle before the session can be created. The brief defines three identification paths: automatic RFID matching, automatic registration plate detection, and manual customer selection. These identification flows are modelled as part of the session-start use case rather than as a separate identification service because they exist specifically to establish the vehicle context for a charging session.

The supported identification paths are:

- **RFID match** — The request includes an RFID identifier. If the RFID is associated with an active vehicle belonging to the customer, that vehicle is selected automatically.
- **Registration plate detection** — The request includes a recognised registration plate. The plate is resolved against the customer's active vehicles only. The plate is not treated as a globally unique vehicle identity because ownership can change over time (Section 5.1).
- **Manual selection** — The customer or client application provides a vehicleId explicitly. If an RFID is included in the request and the selected vehicle does not already have an associated RFID, the RFID is stored against the vehicle to enable future automatic identification.

The physical RFID reader and licence plate recognition integrations are outside the scope of this exercise. There is no live charger/device feed involved (Section 4). However, the API contract, identification rules, and RFID association behaviour are implemented and testable using seeded data.

6. Roles & Authorization Model

Authorization is enforced through three complementary concepts: user type, assigned permissions, and data scope. A role or account type defines the general capabilities of an actor, while permissions and tenant-level rules determine the specific actions and records that actor can access.

The data-scope layer is the critical design decision. Role checks alone are insufficient in a multi-tenant platform because they do not prevent a user from accessing another tenant's resources through manipulated identifiers or directly guessed resource IDs.

Admin

- Global platform scope.
- Can manage vendors, platform configuration, and cross-vendor operational data.
- Can view aggregated income information across vendors.
- Can manage platform-level settings.

Vendor User

- Scoped exclusively to their assigned vendor.
- Can access only that vendor's chargepoints, charging sessions, and related operational data.
- Specific actions are controlled through assigned permissions, such as:
 - `MANAGE_CHARGEPOINTS`
 - `VIEW_INCOME`
 - `MANAGE_VENDOR_USERS`

Customer

- Scoped exclusively to their own account.

- Can access only their own vehicles, charging sessions, and account history.

Authorization enforcement combines:

- **Application-level authorization** using Spring Security method security (for example, `@PreAuthorize`) to validate user capabilities before executing use cases.
- **Repository-level scoping** to ensure queries are constrained to the caller's permitted data scope and to provide defence-in-depth against IDOR-style access attempts.

This ensures that authorization is not dependent solely on API-layer checks. Even if a resource identifier is manipulated, the underlying data access layer still applies the appropriate ownership boundaries.

7. API Surface

Beyond standard per-entity CRUD, the requirements imply a set of business-capability endpoints. These were prioritised over generic CRUD completeness, in line with the brief's guidance to favour judgement over exhaustive coverage:

- POST /vendors — admin onboards a vendor and invites the original vendor user
- POST /vendors/{id}/users — invite additional vendor users
- POST/PUT/DELETE /vendors/{id}/chargepoints — add, update, and remove a vendor's chargepoints
- PUT /vendors/{id}/markup — set the markup applied to a vendor's unit price
- POST /users/{id}/reset-credentials — admin credential reset
- GET /admin/income?vendorId=&from=&to= — income view, optionally filtered by vendor
- GET /vendors/{id}/chargepoints/{cpId}/income?breakdown=day|week|month — chargepoint-level activity, including the current month
- GET /chargepoints/{id}/sessions/report?date= — session report for a chargepoint on a given date
- POST /chargepoints/{id}/sessions — start a charging session; the vehicle is identified by RFID, by registration plate, or by manual customer/vehicle selection, and a newly-supplied RFID is associated with the vehicle when given during a manual selection (see Section 5.5)
- PATCH /sessions/{id}/stop — stop a charging session, recording end time, total energy, total charged, and an error code if the session was partially or fully unsuccessful
- GET /customers/{id}/sessions?month= — session history and totals, including the current partial month
- POST/DELETE /customers/{id}/vehicles — vehicle registration and de-listing
- GET /admin/search?q= — full-text search across sessions, customers, and vehicles

Client-application needs were considered even though no UI was required: list endpoints support pagination and filtering, and monetary/date fields use unambiguous, timezone-explicit representations to avoid client-side misinterpretation.

8. Authentication Approach

A Spring Cloud Gateway configured as a stateless OAuth2 Resource Server sits in front of the backend application and validates incoming JWT access tokens before requests reach business modules. The gateway validates token signature, expiry, and issuer according to the OAuth2 Resource Server contract.

For this exercise, token issuance is implemented within the application rather than delegated to an external Identity Provider. The application maintains user credentials using secure password hashing (Argon2/BCrypt), authenticates users, and issues JWT access tokens. The backend user model stores platform identity and authorization information, including account type (ADMIN, VENDOR_USER, CUSTOMER).

After authentication, backend modules are responsible only for authorization decisions, including role checks, assigned permissions, and tenant/resource scoping (Section 6). Token parsing and validation logic is not duplicated across controllers or domain modules.

Rationale

Separating token validation from business authorization establishes a clear AuthN/AuthZ boundary:

- **Authentication:** Who is the caller?
→ Gateway validates JWT tokens.
- **Authorization:** What is the caller allowed to do?
→ Backend modules evaluate roles, permissions, and ownership rules.

This creates a migration path towards an external OIDC provider without requiring authentication logic to be distributed throughout the application. When a managed identity provider such as Keycloak or Azure AD is introduced, the gateway can be configured to trust the provider's issuer and JWKS endpoint while existing backend authorization rules remain unchanged.

Deferred Alternative

A managed identity provider using OIDC (for example Keycloak) is the preferred long-term approach once the platform requires first-party mobile applications, third-party integrations, social login, enterprise federation, or centralised MFA management.

Operating an Identity Provider introduces additional infrastructure and operational concerns that are not justified for the current exercise scope. The current implementation therefore keeps authentication self-contained while preserving the OAuth2 Resource Server boundary required for a future migration.

This deferral applies only to authentication. The authorization model, including roles, permissions, and tenant scoping described in Section 6, remains owned by the platform and is independent of the chosen identity provider.

9. Full-Text Search Design

Although the requirement describes this capability as "full-text search", the provided example — searching for AUD matching registration plates such as AUD186 and AUD994 — is primarily a partial identifier matching problem rather than traditional document-oriented full-text search.

Conventional PostgreSQL full-text search uses tokenisation and linguistic processing, which is suitable for natural language content but not for structured identifiers such as registration plates, account numbers, or error codes. A value such as AUD186 would normally be treated as a single token, making partial fragment matching ineffective.

Chosen Approach

The implementation uses PostgreSQL's `pg_trgm` extension with GIN trigram indexes on searchable identifier fields, including:

- vehicle registration plates
- customer account numbers
- session error codes

The search capability is exposed through a single search endpoint that performs indexed partial matching across supported entities.

This approach provides efficient substring and similarity matching at the scale expected for the exercise without introducing a separate search infrastructure component.

Alternatives Considered

SQL LIKE '%term%'

Rejected as the primary approach because although it provides correct substring matching, it cannot efficiently use normal B-tree indexes and performance degrades as data volume increases.

It remains a valid fallback option for environments where PostgreSQL extensions cannot be enabled.

Elasticsearch / OpenSearch

Deferred because although it is a natural choice for large-scale search workloads involving:

- complex relevance ranking
- large document collections
- advanced filtering and aggregation
- cross-system search

It introduces additional operational complexity that is not justified for the current application scale.

The current design keeps search close to the transactional data while leaving a clear migration path to a dedicated search platform if future requirements exceed PostgreSQL's capabilities.

10. Data Architecture & Module Boundaries

Even within a single deployable monolith, data ownership determines whether the system remains modular and extractable or gradually evolves into a tightly coupled distributed system. Two decisions are central to maintaining these boundaries.

10.1 Schema-per-Module in a Single PostgreSQL Instance

The database is organised into logical schemas aligned with the business module boundaries defined in Section 3.1. For example, Station Management owns the station schema, Pricing owns the pricing schema, and Session Management owns the session schema.

Each module owns and manages its own persistence model. A module does not directly query or modify another module's tables, even though all modules currently run within the same PostgreSQL instance. Cross-module data access occurs through explicit application interfaces or domain events rather than direct database joins.

This approach introduces minimal operational overhead for the current modular monolith: there remains a single database instance, connection pool, backup strategy, and deployment process. However, it preserves clear ownership boundaries and prevents accidental coupling through shared tables.

When future requirements justify extracting modules into independent services, these boundaries provide a stronger migration path because each module's data ownership is already explicit. Extraction would still require addressing distributed transactions, API contracts, and data migration concerns, but the system avoids the larger challenge of separating tightly shared database structures.

10.2 Module Boundary Enforcement

The package structure mirrors the business module boundaries, and those boundaries are enforced automatically at build time. Architectural rules (for example, using ArchUnit) prevent one module from importing another module's internal implementation details, such as domain entities, repositories, or infrastructure classes.

Cross-module interaction occurs only through explicit contracts exposed by the owning module, such as application service interfaces or published domain events. A module may depend on another module's public API, but it cannot bypass that boundary by accessing internal state or persistence structures directly.

This follows the dependency discipline of Hexagonal Architecture and Ports-and-Adapters design. Each module contains its own domain logic behind well-defined ports, while adapters handle external concerns such as persistence, messaging, and API exposure.

Because communication is expressed through module contracts rather than implementation details, replacing an in-process call with a remote interaction (such as REST, gRPC, or messaging) becomes an adapter-level change rather than requiring business logic rewrites.

10.3 Concrete Modules in This Exercise

To keep the exercise implementation aligned with the vision decomposition described in Section 3.1, the application is organised into five explicit business modules rather than a generic layered architecture.

This is a deliberate scope decision: the system is deployed as a single modular monolith, not as separate services, but each module has a defined responsibility boundary and corresponds closely to a future service boundary where applicable.

Module	Owns	Vision Service Alignment
Identity & Access	User accounts, roles, permissions, and credential management. Authentication is implemented within this module for the exercise because no external Identity Provider is required.	Matches the Identity & Access authorization responsibility directly and provides a temporary authentication implementation that can later be replaced by an external IdP such as Keycloak.
Station Management	Vendor, VendorUser, and Chargepoint entities, including vendor base charging rates.	Directly matches the Station Management service boundary.
Pricing	Admin-configured vendor markup and calculation of effective charging rates by combining vendor base rates and markup. Invoicing and billing statements are explicitly out of scope (Section 12.3).	Covers the pricing responsibility from the future Pricing & Billing service while keeping pricing rules independently owned.
Session Management	ChargingSession lifecycle data. Session Management requests effective rates from Pricing and snapshots the calculated rate when a session starts.	Matches Session Management directly. Customer and vehicle management are included here because they exist primarily to support session workflows within this exercise scope.
Reporting & Search capability	Income views, chargepoint/session reports, and PostgreSQL trigram-based search	Represents a simplified analytics capability over operational modules rather than a separate transactional domain.

The following vision components are intentionally not implemented in this exercise:

- **Payment Processing**
- **Notification**
- **Device Gateway**

These are not simplified versions of modules; they are explicitly outside the implementation scope because the exercise does not require payment integration, messaging workflows, or live charger/device communication.

Pricing is intentionally implemented as its own module rather than embedded inside Session Management. Session Management owns the lifecycle of charging activity, while Pricing owns the calculation of charging rates. This separation keeps the responsibilities independently testable and preserves a clear extraction path if pricing becomes an independent service in the future.

11. Build Order

The implementation order follows dependency direction: establish the architectural foundation first, then build business capabilities on top of stable module boundaries.

1. **Scaffold the application foundation**
 - Create the Spring Boot project structure.
 - Configure WebFlux/MVC, JPA, Security, PostgreSQL, schema migrations, and local development environment.
 - Establish module package boundaries and logical database schemas from the beginning.
2. **Implement the authentication and authorization foundation**
 - Create the user and permission model.
 - Implement JWT authentication flow and OAuth2 Resource Server validation.
 - Configure security boundaries between authentication, authorization, and tenant/resource scoping.
 - Add initial security tests before exposing business APIs.
3. **Implement core domain modules**
 - Build vendor, vendor user, chargepoint, customer, and vehicle capabilities.
 - Apply tenant-scoping rules from the first implementation rather than adding isolation later.
 - Add module-level tests covering ownership and permission boundaries.
4. **Implement charging session lifecycle**
 - Create session start and stop flows.
 - Implement vehicle identification rules.
 - Add pricing rate snapshotting, month bucketing, and monetary precision handling.
 - Verify session behaviour through domain and integration tests.
5. **Implement reporting capabilities**
 - Add income summaries, chargepoint activity reports, and session reporting endpoints.
 - Ensure reporting respects the same authorization and tenant-scoping rules as operational APIs.
6. **Implement search capability**
 - Add PostgreSQL pg_trgm extension support.
 - Create GIN trigram indexes.
 - Implement indexed partial matching through the search endpoint.
7. **Complete delivery and operational concerns**
 - Add OpenAPI documentation.
 - Create Dockerfile and docker-compose configuration.
 - Add automated tests covering critical business rules, especially pricing calculations, authorization, and tenant isolation.
8. **Maintain architectural documentation**
 - Keep the architecture decision document aligned with implementation decisions.
 - Document deviations, trade-offs, and future extraction considerations as the system evolves.

12. What Was Built, Simplified, and Deferred

Consistent with the brief's preference for well-reasoned trade-offs over completeness, this section makes explicit what was fully implemented, what was implemented in a deliberately narrower form, and what was left out entirely with a stated production path.

12.1 Built (fully implemented)

Area	What	Why it made the cut
Core entity model	AdminUser, Vendor, VendorUser, Chargepoint, CustomerUser, Vehicle, ChargingSession with correct relationships and constraints	Foundation everything else depends on
Multi-tenant CRUD	Full CRUD for vendors, chargepoints, customers, vehicles, with tenant-scoped repository queries	Demonstrates the core architectural judgement call (row-level isolation)
Charging session recording	Session creation with vehicle identification (RFID, plate, or manual selection), rate-snapshotting, month bucketing, energy/total calculations	Transactional heart of the domain — has to be correct
Authentication	Spring Cloud Gateway as OAuth2 Resource Server (JWT validation at the edge), role model (ADMIN / VENDOR USER / CUSTOMER), password hashing	Explicit deliverable — REST API with authentication
Full-text search endpoint	pg_trgm + GIN index search across plate, account number, error code, with partial-match support	Explicit, testable requirement with a concrete worked example
API documentation	Swagger / OpenAPI	Explicit deliverable
Dockerised solution + seed data	Docker Compose (app + Postgres), SQL seed script on startup	Explicit deliverable
Core automated tests	Unit tests for pricing/markup math and tenant-scoping logic; integration tests on the trickiest endpoints	Highest-risk areas for silent breakage

12.2 Simplified (built, but deliberately narrower than a full production version)

Area	What was simplified	Why
Vendor user permissions	A small fixed permission set (e.g. manage chargepoints, view income, manage vendor users) rather than a configurable permission-builder	Proves the extensible data-model pattern without building unrequested admin tooling
Reporting breakdowns	Day/week/month income views implemented; custom date ranges and YoY comparisons left out	Covers what's explicitly requested; open-ended reporting has no fixed spec
Search	Trigram substring search across the three named fields only, not a general-purpose search-everything endpoint	Matches the exact requirement given; broader scope wasn't asked for
Validation & error handling	Solid validation where correctness risk is highest (money, session data); lighter elsewhere	Time-boxing judgement — depth where it matters most

12.3 Deferred (explicitly not built, with a stated production path)

Area	Deferred to	Why deferred
------	-------------	--------------

Billing / invoicing	Out of scope entirely	Brief explicitly states billing is handled elsewhere
Managed identity provider (OIDC)	Documented as the production path (e.g. Keycloak)	No external client ecosystem yet to justify the infrastructure; resource-server JWT validation is proportionate now
Elasticsearch / OpenSearch	Documented as the natural next step if search volume/relevance needs grow	pg_trgm fully satisfies the stated requirement at this scale
Email delivery (invites, credential resets)	Endpoint stubbed / logged; real delivery via SES/SendGrid noted	Delivery integration is an ops/vendor decision, not an architecture decision
Rate limiting / API gateway	Noted as a platform-layer concern	Belongs at the infrastructure/gateway level in production
Full audit logging	Noted as required for production	Valuable but orthogonal to proving the core design; would duplicate effort across every write endpoint
CI/CD pipeline	Noted as next step	Not part of the stated deliverables

12.4 The Underlying Logic

The scope decisions throughout this document follow a consistent principle: implement what is required for business correctness today, preserve the right architectural boundaries for tomorrow, and avoid introducing operational complexity before there is a requirement to justify it.

Everything in **Built** either directly satisfies an explicit deliverable or provides a foundation required for correctness, such as monetary precision, tenant isolation, authentication, authorization, and historical data integrity.

Everything in **Simplified** implements the essential business pattern while keeping the solution proportional to the exercise scope. These areas demonstrate the intended architectural approach without introducing unnecessary flexibility, infrastructure, or complexity that has no current consumer.

Everything in **Deferred** represents a capability that becomes valuable as the platform grows but is not required at the current stage. These decisions are driven by scale, operational ownership, or integration needs rather than correctness concerns. They remain documented as future evolution paths rather than ignored requirements.

This distinction between premature complexity and proportionate design is the common thread behind the architectural trade-offs in this document: build the capabilities that matter now, preserve the boundaries that enable future growth, and defer infrastructure investment until the problem it solves actually exists.

13. Non-Functional & Productionisation Considerations

The MVP implementation focuses on the non-functional requirements that directly affect correctness and security, while documenting operational capabilities that become necessary as the platform scales.

Multi-Tenant Data Isolation

Multi-tenant isolation is treated as a first-class security concern. Access control is enforced at multiple layers:

- application authorization checks validate user permissions and ownership rules;
- repository/query boundaries ensure data access is constrained to the caller's permitted scope.

Tenant isolation is therefore not dependent solely on API-layer validation, reducing the risk of IDOR-style access through manipulated resource identifiers.

Monetary Precision

All monetary calculations use deterministic precision rules throughout the system. Values are represented using integer-based monetary units or fixed-scale BigDecimal values with an explicit rounding policy.

The same precision rules apply across persistence, calculations, and API responses to prevent rounding drift in markup calculation, charging totals, and reporting.

Timezone Handling

All timestamps follow an explicit timezone policy:

- timestamps are stored in UTC;
- reporting boundaries are calculated using the relevant business timezone;
- vendor-local timezone rules are applied when displaying monthly summaries and "current partial month" reports.

This prevents subtle date-boundary errors around month transitions and daylight-saving changes.

Idempotency

Charging session operations must tolerate retries caused by unreliable charger communication or network interruptions. Session creation should support idempotent behaviour by using a stable external identifier or idempotency key so that repeated requests do not create duplicate charging sessions.

This is particularly important when integrating with physical charging infrastructure where message delivery may be retried.

Observability

The platform adopts lightweight operational visibility from the beginning:

- structured application logs;
- correlation/request identifiers;
- health checks;
- basic application metrics.

A full observability platform using distributed tracing, centralized logging, and metrics aggregation (for example, OpenTelemetry with Grafana-based tooling) is deferred until operational scale requires it.

The principle is to implement the low-cost foundations early while avoiding premature operational infrastructure.

Backup & Recovery

For the exercise environment, managed PostgreSQL backups combined with an append-only audit record of important business events are sufficient.

A production deployment handling real transactions and payment data would require stronger recovery guarantees, including:

- point-in-time recovery;
- retention policies;
- tested restore procedures;
- potentially cross-region replication.

Rate Limiting

Production deployments would enforce rate limiting at the API gateway layer. This capability is intentionally not implemented in the exercise, but the architecture maintains a single ingress boundary where policies such as throttling, request quotas, and abuse protection can be introduced without modifying business modules.

Database Migration Strategy

The exercise uses SQL-based startup initialization for convenience and repeatability in development environments.

A production deployment would use a managed migration framework such as Flyway or Liquibase to provide:

- version-controlled schema evolution;
- repeatable deployments;
- migration auditing;
- controlled rollback strategies where appropriate.

14. Architecture Decision Log: Reconciling Vision-Scale Decisions

A separate architecture review explored vision-scale decisions across protocol integration, service decomposition, data ownership, transactions, authentication, observability, deployment, and operational concerns.

These decisions did not change the implementation scope of this exercise. Instead, they validated the modular-monolith-first approach and highlighted several architectural refinements that improve the evolution path without adding unnecessary complexity.

The following table reconciles each vision-scale decision against the current implementation baseline, making the relationship between the MVP design and the future platform architecture explicit.

Decision Area	Vision-Scale / MVP Recommendation	Status vs. This Baseline
Device communication protocol	OCPP 1.6J over WebSocket, behind a protocol abstraction layer; migrate to 2.0.1 later	New scope — not built; this exercise has no live device gateway (Section 4)
Service decomposition	DDD boundaries: Identity, Station, Session, Pricing & Billing, Payment, Notification, Device Gateway	Aligned — mirrors this document's module split (Sections 3.1, 15)
Deployment shape for v1	Modular monolith with enforced module boundaries, not microservices	Aligned — this is the exercise's core architectural choice (Section 4, 10.2)
Data ownership	Single PostgreSQL, schema-per-module, no cross-schema joins	Adopted as refinement — formalised in Section 10.1
Inter-module communication	Hexagonal ports/adapters; in-process now, swappable to REST/gRPC later	Adopted as refinement — formalised in Section 10.2
Cross-module transactions	Lightweight workflow orchestrator with explicit compensation; Temporal later if needed	New scope — this exercise has a single-service transaction (session create + price calc); no multi-step saga exists yet
External API contract	REST + OpenAPI for commands/queries; SSE for real-time updates	Partially aligned — REST + OpenAPI built (Section 7); SSE deferred, no real-time push requirement in this exercise
Authentication	OAuth2/OIDC resource-server pattern; avoid standing up an IdP until justified	Aligned — matches this document's authentication decision exactly (Section 8)
Device Gateway isolation	Dedicated module, isolated from Station Mgmt, simple stack for v1	New scope — no device gateway in this exercise; noted as a future module (Section 15)
Billing & payment	Separate Billing and Payment modules; tokenised payment methods; settlement decoupled from session lifecycle	Directionally aligned — billing/payment explicitly deferred here (Section 12.3) for the same reason: keep it out of the core transaction path
Observability	OTel-style instrumentation from day one; defer the full backend stack	Aligned — matches Section 13's observability stance exactly
Deployment infrastructure	Containerise from day one; defer Kubernetes/GitOps until scale justifies it	Aligned — this exercise ships Docker Compose, not Kubernetes
Firmware management	Metadata-only in Station Mgmt for v1; defer staged rollout entirely	New scope — firmware is outside this exercise's entity model; noted for future extension
Multi-vendor isolation	Vendor-scoped data with logical isolation, not physical multi-tenancy	Aligned — this is exactly this document's tenancy model (Section 6)
Session lifecycle modelling	Simplified lifecycle for v1; protocol state and business state kept separate	Partially aligned — this exercise models session data, not a full protocol-vs-business

		state machine (no live device feed to drive one)
Rate limiting	Basic protection at the gateway layer for v1; defer granular per-domain limits	Aligned — matches Section 13's rate-limiting stance
Testing strategy	Strong unit + integration tests; defer formal contract testing until modules are extracted	Aligned — matches this document's testing scope (Section 12.1)
Backup & disaster recovery	Managed backups + immutable audit log for v1; defer full PITR and DR drills	Aligned — matches Section 13's backup stance exactly
Multi-region deployment	Single region, multi-AZ for v1; multi-region only when latency/compliance demands it	Out of scope — not applicable at this exercise's scale; no change needed
CI/CD	Simple build/test/scan/deploy pipeline; defer GitOps and progressive delivery	New scope — CI/CD noted as future work (Section 12.3), consistent with this recommendation

Two important architectural refinements emerged from this reconciliation and were incorporated into the baseline design:

1. **Schema-per-module data ownership (Section 10.1)**
2. **Explicit module-boundary enforcement (Section 10.2)**

Both introduce minimal implementation cost at the current scale while significantly improving maintainability, testability, and future extraction options.

This reflects the central principle behind the reconciliation process: introduce architectural discipline where it protects future evolution, but defer infrastructure and operational complexity until the platform has a real requirement for it.

15. Adapting the Exercise Design to the Platform Vision

This section makes the evolution path concrete: which of the exercise's five modules (Section 10.3) map directly onto the target service decomposition, and which vision services are net-new builds triggered only by real demand.

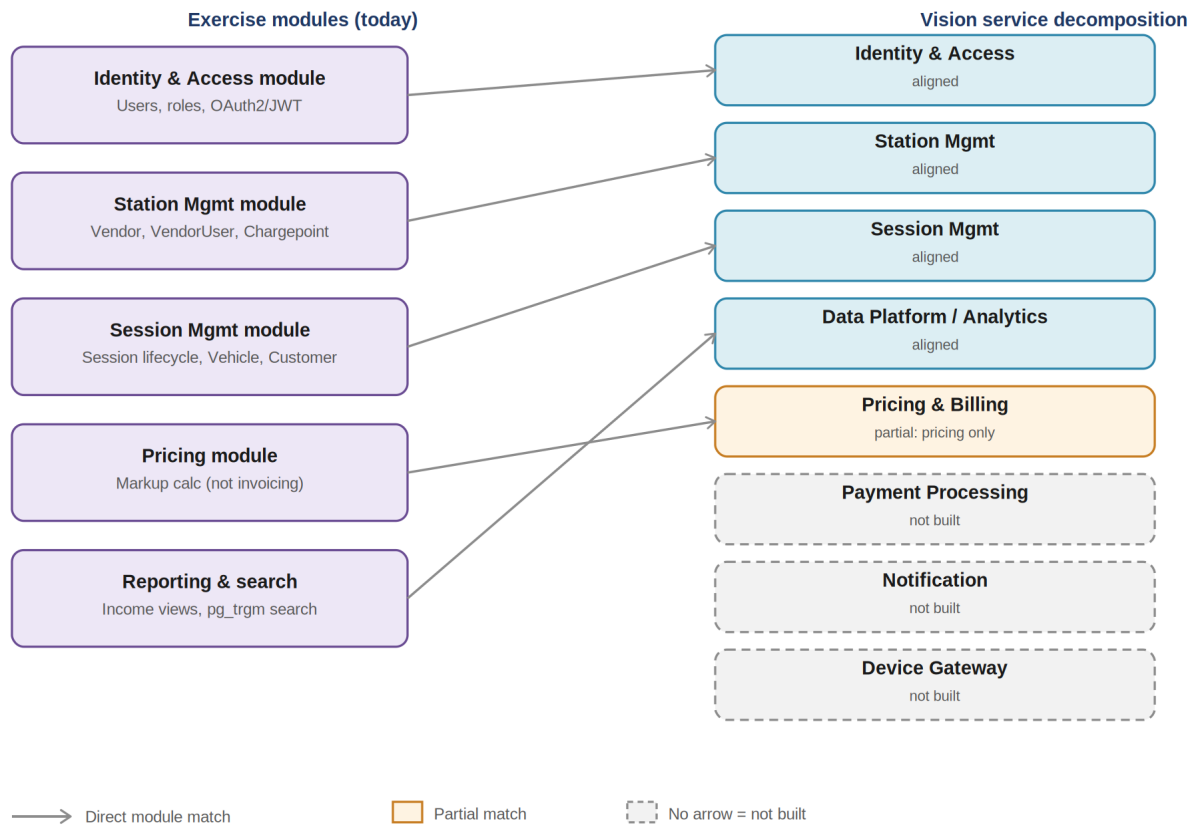


Figure 4 — Exercise modules mapped onto the target service decomposition. Every built module maps one-to-one onto its vision service; boxes with no arrow have no equivalent in the exercise at all.

15.1 Module → Vision Service Mapping

Exercise Module	Vision Service	Match	Note
Identity & Access module	Identity & Access	Direct	One-to-one; same responsibility, same data
Station Mgmt module	Station Mgmt	Direct	One-to-one; same responsibility, same data
Pricing module	Pricing & Billing	Partial (pricing only)	Rate/markup calculation is built and kept separate from Session Mgmt; invoicing and billing statements are not built, per Section 12.3
Session Mgmt module	Session Mgmt	Direct	One-to-one for session lifecycle and recording; calls the Pricing module rather than calculating rates itself
Reporting & Search	Data Platform / Analytics	Direct (as a capability, not a module)	Implemented as read/reporting logic over the other modules' data rather than a sixth standalone module
— none —	Payment Processing	Not built	Explicitly out of scope; billing is handled elsewhere per the brief

— none —	Notification	Not built	No client application in this exercise to notify
— none —	Device Gateway	Not built	No live OCPP/device feed; chargepoints are managed via CRUD and seed data only

Every module that exists in this exercise maps onto its vision service — directly for Identity & Access, Station Mgmt, and Session Mgmt, and partially for Pricing, which covers rate and markup calculation but not invoicing or billing statements. Pricing is deliberately kept as its own module rather than folded into Session Mgmt, so that pricing rules can grow in complexity (tiered rates, promotions, time-of-use pricing) without touching session-lifecycle code, and so that extending it to cover full billing later is additive rather than a rewrite. The remaining gap is Payment Processing, Notification, and Device Gateway, which simply do not exist in this exercise: this remains, and must remain, a single modular monolith of five modules — the right shape for its scope, not a precursor to prematurely splitting services.

15.2 Actor Mapping

Vision Actor	Maps To	Status
Administrator / network operator	AdminUser	Built
Charging station vendor	Vendor + VendorUser	Built
Individual driver	CustomerUser + Vehicle	Built
Business / fleet operator	CustomerUser with multiple vehicles	Data model supports it; no fleet-specific tooling built
Charging device (IoT)	Chargepoint	Modelled; no live OCPP/MQTT feed — data arrives via seeded/simulated sessions
Vehicle	Vehicle	Modelled; no live telematics API integration

15.3 Domain Scope: Built vs. Vision

Domain	Status Now	What Would Trigger Building It
IAM / RBAC	Built	—
Station management	Built	—
Charging session recording	Built	—
Basic reporting / income views	Built	—
Real OCPP / Device Gateway	Not built — brief asks only for a data model plus seed data	A real hardware fleet needing live BootNotification / Heartbeat / MeterValues ingestion
Pricing (rate/markup calculation)	Built	—
Payment Processing	Not built — brief states billing is handled elsewhere	A real payments-provider integration decision, outside this service's boundary
Dynamic / demand pricing	Not built	Brief specifies only a flat unit price plus vendor markup

Notification service	Not built — no UI required	A product decision once a client app exists
Fleet optimisation, predictive maintenance, smart-grid balancing, AI features	Vision only	Real scale and real IoT telemetry volume, realistically well beyond this exercise's horizon

15.4 How This Connects to the Extraction Path

The future platform vision does not change the recommendation to keep the current implementation as a modular monolith. Instead, it validates and clarifies the future extraction boundaries already established through the module design.

The strongest early extraction candidate is the **Device Gateway** once real OCPP traffic exists. Unlike customer- and vendor-facing business flows, device communication has a fundamentally different operational profile:

- long-lived WebSocket connections;
- protocol-specific concerns;
- high-volume telemetry ingestion;
- different availability and scaling requirements.

Keeping this responsibility isolated from Station and Session Management today prevents device concerns from becoming embedded inside transactional business workflows.

Similarly, maintaining **Pricing** as an independent module rather than embedding rate calculation inside Session Management preserves a clean evolution path. Extending pricing capabilities into invoicing and billing can therefore be additive work on top of an existing boundary rather than requiring extraction from tightly coupled session logic.

Billing, Payment Processing, and future capabilities such as fleet optimisation remain separate concerns. They should integrate through well-defined interfaces because they introduce independent compliance requirements, operational characteristics, and data ownership models.

The schema-per-module and ports/adapters discipline described in Section 10 significantly reduces future extraction complexity. Moving towards independent services would still require deliberate work around API contracts, data migration, distributed transactions, and operational ownership, but the system avoids the more difficult problem of separating deeply shared business logic and database structures.

The platform vision therefore serves as a roadmap for future evolution rather than a justification for premature complexity. The architecture establishes the boundaries needed for future growth while keeping the current implementation proportional to today's requirements.

16. Requirements Traceability Matrix

Sections 5 through 9 describe the design; this section checks that design directly against every bullet in the original brief, bullet by bullet, so coverage is verifiable rather than assumed. “Built” means implemented and testable against seed data; “Assumption” means the brief left the point underspecified and a sensible default was made explicit elsewhere in this document; “Out of scope” means the brief itself excludes it.

16.1 Setup

Requirement	Status	Covered In
Spring Initializr project; manage entities in a relational database; access via REST APIs with authentication	Built	Sections 4, 7, 8

16.2 Admin Users

Requirement	Status	Covered In
Properties: name, email	Built	Section 5
Sign up vendors, including inviting the original vendor user	Built	Section 7 (POST /vendors)
View income over a period, optionally filtered by vendor	Built	Section 7 (GET /admin/income)
Set the markup added to the vendor's unit price	Built	Section 7 (PUT /vendors/{id}/markup); Section 10.3 (Pricing module)
Reset credentials for users where appropriate	Built	Section 7 (POST /users/{id}/reset-credentials)

16.3 Vendors

Requirement	Status	Covered In
Properties: name, account number	Built	Section 5
Have one or more users who manage vendor-owned resources per assigned permissions	Built	Sections 5.4, 6
Own chargepoints and can add, update, and remove them	Built	Section 7 (chargepoint CRUD)
Insights into income and charging activity, including the current month and breakdowns by chargepoint over days/weeks/months	Built	Section 7 (GET /vendors/{id}/chargepoints/{cpId}/income?breakdown=)
Can generate reports of charging sessions for a chargepoint on a given date	Built	Section 7 (GET /chargepoints/{id}/sessions/report?date=)

16.4 Vendor Users

Requirement	Status	Covered In
Properties: name, email, vendor	Built	Section 5
Belong to exactly one vendor	Built	Section 5
Can manage vendor-owned resources according to their assigned permissions	Built	Sections 5.4, 6

16.5 Chargepoints

Requirement	Status	Covered In
Properties: unique identifier, group label, unit price, availability, vendor	Built	Section 5
Owned by a vendor	Built	Section 5
Unique identifier displayed on the physical chargepoint for support purposes	Built	Section 5
May have a group label to identify multiple chargepoints belonging together	Built	Section 5

Unit price in tenths of cents	Built	Section 5.2 (monetary precision)
May be temporarily unavailable	Built	Section 5 (availability field)

16.6 Customer Users

Requirement	Status	Covered In
Properties: name, email, account number, phone number	Built	Section 5
Register with the platform	Built	Standard CRUD, Section 7 intro
Perform charging sessions using the marked-up unit price	Built	Section 5.5; Section 10.3 (Pricing module)
View charging session history and totals by month, including the current partial month	Built	Section 7 (GET /customers/{id}/sessions?month=)
Billing is handled elsewhere	Out of scope (by design)	Section 12.3
Own one or more vehicles and maintain them themselves	Built	Section 5; Section 7 (vehicle CRUD)
Can de-list vehicles (e.g. following disposal or sale)	Built	Section 7 (DELETE /customers/{id}/vehicles); Section 5.1 (soft delete)
If a vehicle is later owned by another customer, it may be registered again with the same plate	Built (assumption documented)	Section 5.1 (vehicle identity over time)

16.7 Vehicles

Requirement	Status	Covered In
Properties: registration plate, RFID number, customer	Built	Section 5
Owned by a customer	Built	Section 5
Can be identified automatically by RFID or plate detection	Built (API + data model); detection hardware itself out of scope	Section 5.5
Can also be selected manually by the customer	Built	Section 5.5
If an RFID is available during a manual selection, it may be associated with the vehicle for future identification	Built	Section 5.5

16.8 Charging Sessions

Requirement	Status	Covered In
Properties: start, end, vehicle, chargepoint, marked-up unit rate, error code, total energy, total charged	Built	Section 5
Belong to the month in which they start	Built	Section 5.3 (month bucketing)
Use the unit price applicable at the session start time	Built	Section 5 (rate snapshot decision)
Record an error code if partially or fully unsuccessful; otherwise empty	Built	Section 5; Section 7 (PATCH /sessions/{id}/stop)
Record the total energy delivered (kWh)	Built	Section 5

Record the total amount charged to the customer	Built	Section 5
---	-------	-----------

16.9 REST API

Requirement	Status	Covered In
REST API with authentication and API documentation (Swagger or equivalent)	Built	Sections 7, 8, 12.1
API designed for the entity model's capabilities, considering client-application needs	Built	Section 7
Authentication approach selected and explained	Built	Section 8

16.10 Full-Text Search

Requirement	Status	Covered In
Endpoint for administrators to search sessions using session, customer, or vehicle information	Built	Section 9
Search supports partial matches (e.g. “AUD” matches “AUD186” and “AUD994”)	Built	Section 9 (pg_trgm + GIN index)

Net result: every capability bullet in the brief is built, with two explicit, brief-sanctioned exclusions (billing/invoicing itself, and physical RFID/plate-detection hardware, since there is no live device feed to detect from) and one documented assumption (vehicle re-registration after resale). No requirement was silently dropped — the two gaps this audit surfaced against an earlier draft (session start/stop endpoints and vehicle identification handling) have been folded into Sections 5.5 and 7 above rather than left implicit.